

CarAI

Genetic-Algorithm Optimisation of a Racing Line

Oliwier Zasadni & Mikołaj Kimak

Artificial Intelligence / Evolutionary Computation
Silesian University of Technology (Politechnika Śląska)

The Problem: Finding the Fastest Racing Line

A 2-D driving game where an AI learns the optimal line around any hand-drawn circuit.

- Objective: minimise lap time around a closed track.
- Hard constraint: the car must stay on the driveable surface (off-track = invalid).
- Search space: the lateral position of the line at every point of the lap - large, continuous, non-convex.
- Why it is a good AI problem: every candidate solution is a real racing line that can be both computed (a lap-time estimate) and watched (replayed in the game).

Why a Genetic Algorithm?

- Analytical / optimal control (minimum-curvature, QP)
 - Exact, but needs a smooth differentiable, convex model - fails for an arbitrary hand-drawn track.
- Reinforcement learning
 - Learns control end-to-end, but sample-inefficient, hard to tune, and produces a black-box policy.
- Genetic algorithm over the racing line → **chosen**
 - Gradient-free, robust to a penalty-based fitness function, and the genome is the racing line - directly interpretable and drawable.
 - Each evolutionary operator (selection, crossover, mutation) is observable - ideal for a didactic project.

How the Line is Encoded

- The track centreline is split into $N = 50$ evenly-spaced cross-sections.
- A chromosome = 50 real numbers in $[0, 1]$; gene d_i is the lateral offset at section i .
 - 0 = inner track edge, 1 = outer track edge.
- The 50 control points are interpolated by a periodic cubic spline into a smooth closed trajectory of 150 points.
- A physics model then estimates the lap time of that trajectory - this is the fitness.
- Result: a fixed-size, low-dimensional genome that can still describe any smooth line inside the corridor.

Fitness: a Physics-Based Lap-Time Estimate

Lower is better. Each chromosome is scored by this pipeline (in metres):

- Curvature at each point (Menger formula) $\rightarrow$ corner radius R .
- Corner-speed limit from the friction circle: $v_{\max} = \sqrt{\mu \cdot g \cdot R}$, $\mu = 0.7$.
- Feasible speed profile: forward + backward passes enforce realistic acceleration / braking limits.
- Lap time = Σ segment length / segment speed.
- Boundary penalty: 500 s per metre off-corridor + 1000 s per off-track point - any excursion is bred out within a few generations.

The Genetic Operators

- Selection - tournament (size 5): mild pressure that preserves diversity.
- Crossover - three variants compared in the experiments:
 - Single-point · Multi-point (3 cuts) · Uniform (per-gene swap).
- Mutation - Gaussian: $d_i += N(0, \sigma)$, $\sigma = 0.05$, per-gene probability = mutation rate.
- Elitism - the best 2 individuals are copied unchanged each generation.
- Defaults: population 100, 200 generations, mutation rate 0.10.

System Architecture

- Godot 4.6 game - draws the circuit, runs car physics, telemetry, and replays the AI line.
- Python GA - evolves the racing line against the physics-based fitness function.
- Coupling via JSON files on disk:
 - Godot → track_data.json (centreline, corridor, boundaries).
 - Python → current_best.json every generation (live preview, polled every 0.5 s).
 - Python → best_line.json on completion (waypoint commands the AI car replays).
- The AI car drives through the same physics engine as the human - only throttle & steering are written, never position.

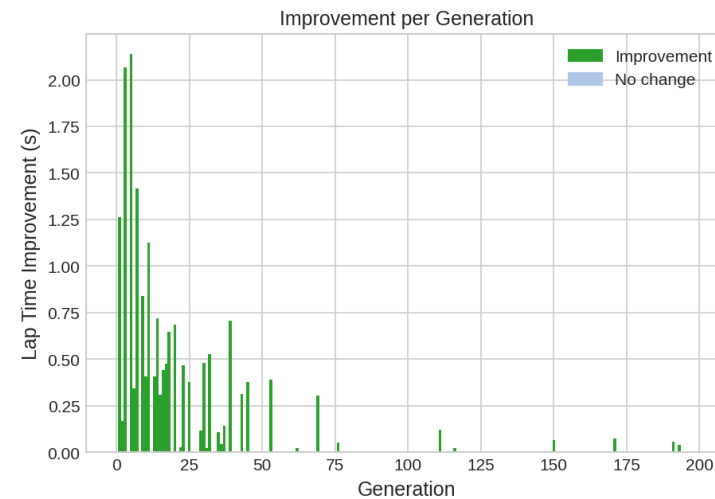
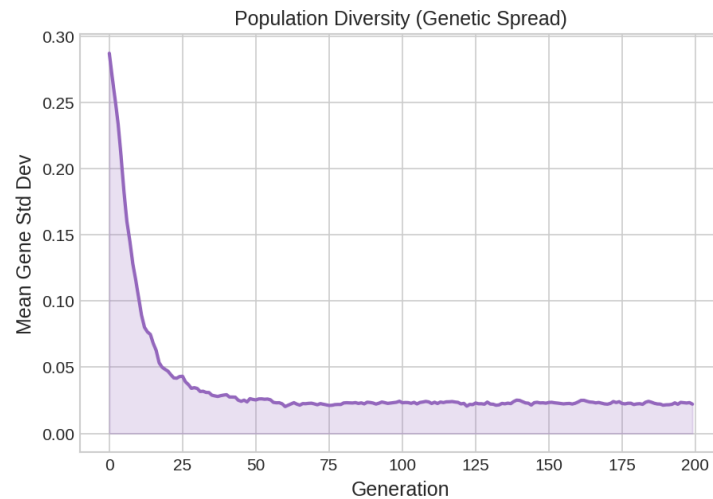
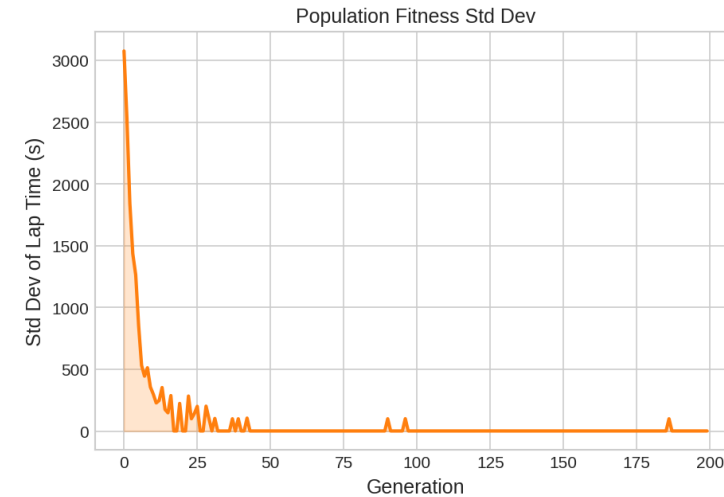
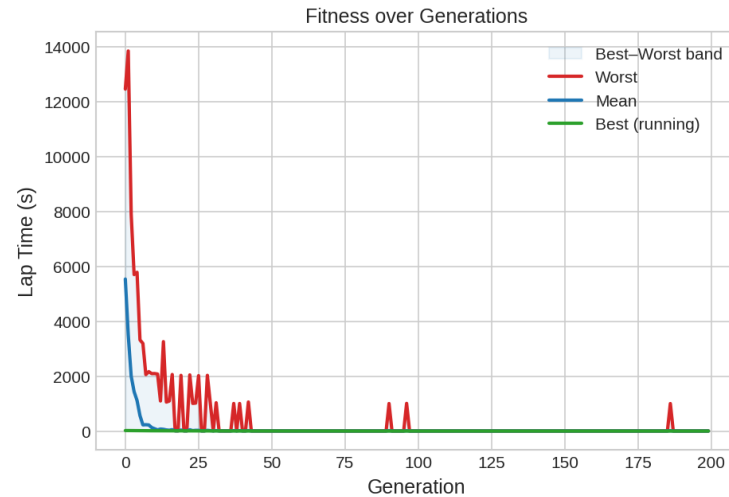
Experiments: What We Measured

Three parameter studies on the same circuit; each config run with 3 seeds [42, 123, 7].

- Set A - Crossover operator: single-point vs multi-point vs uniform.
- Set B - Mutation rate: 0.01, 0.05, 0.10, 0.20, 0.40.
- Set C - Population size: 20, 50, 100, 200.
- Tracked: final lap time (quality), convergence speed, and population diversity.
- Shaded bands = spread across seeds; error bars = ± 1 standard deviation.

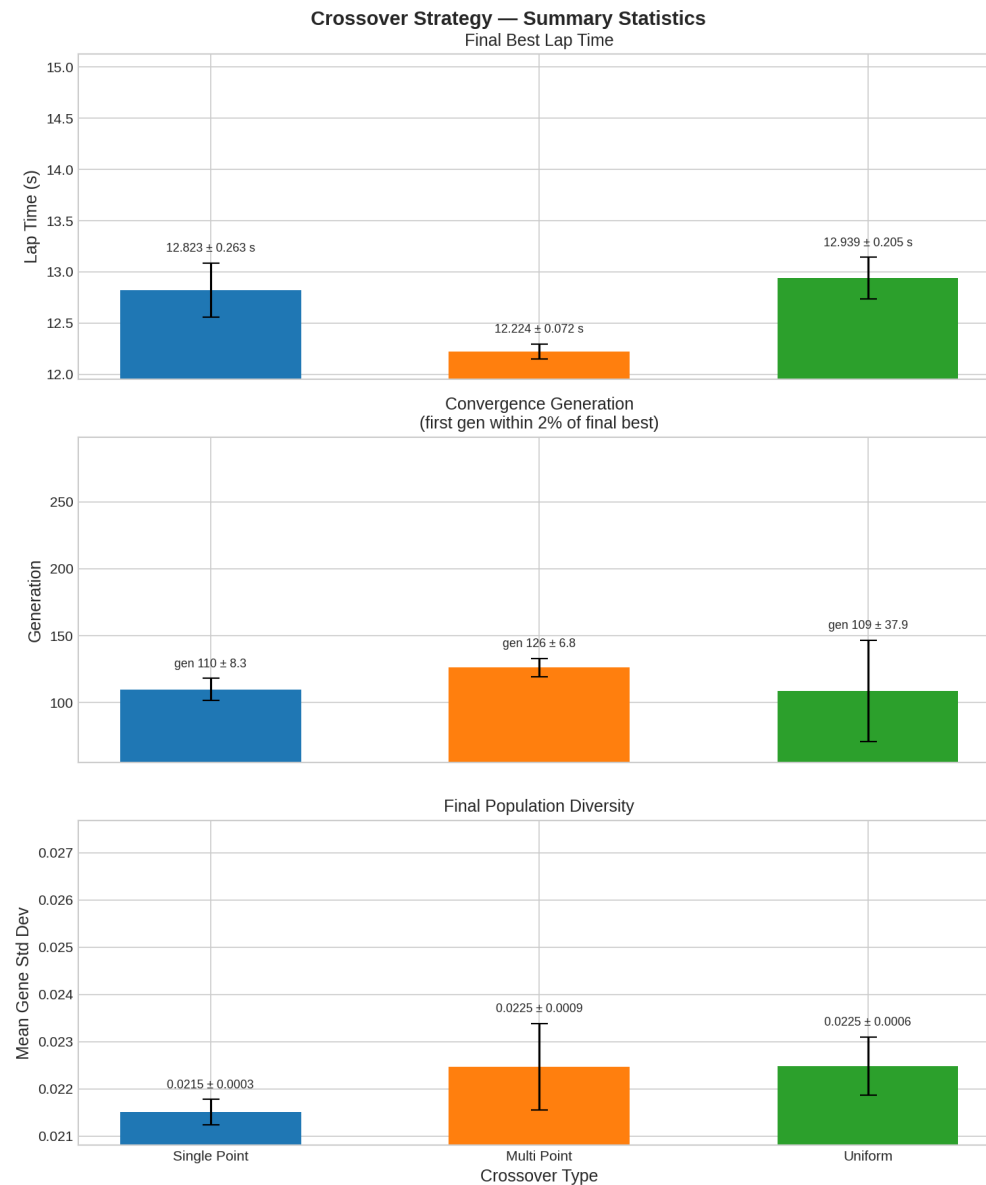
Overall GA Behaviour

GA Performance Dashboard — Default Configuration



Almost all improvement happens in the first ~50 generations; off-track penalties are eliminated by ~gen 40; diversity falls then plateaus.

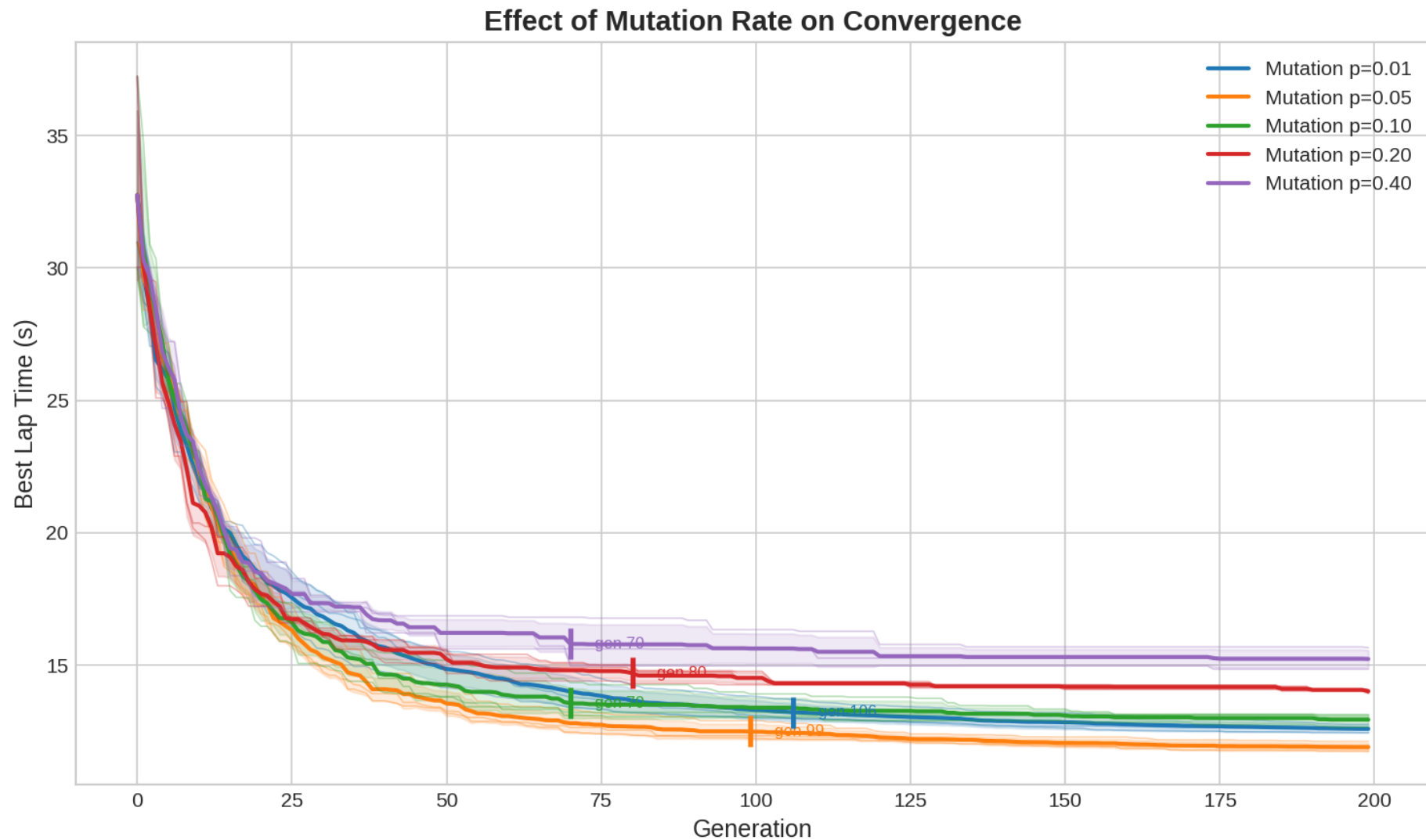
Result - Crossover Operator



Multi-point wins

- Multi-point: 12.224 ± 0.072 s - best & most consistent.
- Single-point: 12.823 ± 0.263 s.
- Uniform: 12.939 ± 0.205 s.
- Multi-point recombines contiguous track segments, preserving good cornering arcs.
- Takeaway: use multi-point for this spatially-structured genome.

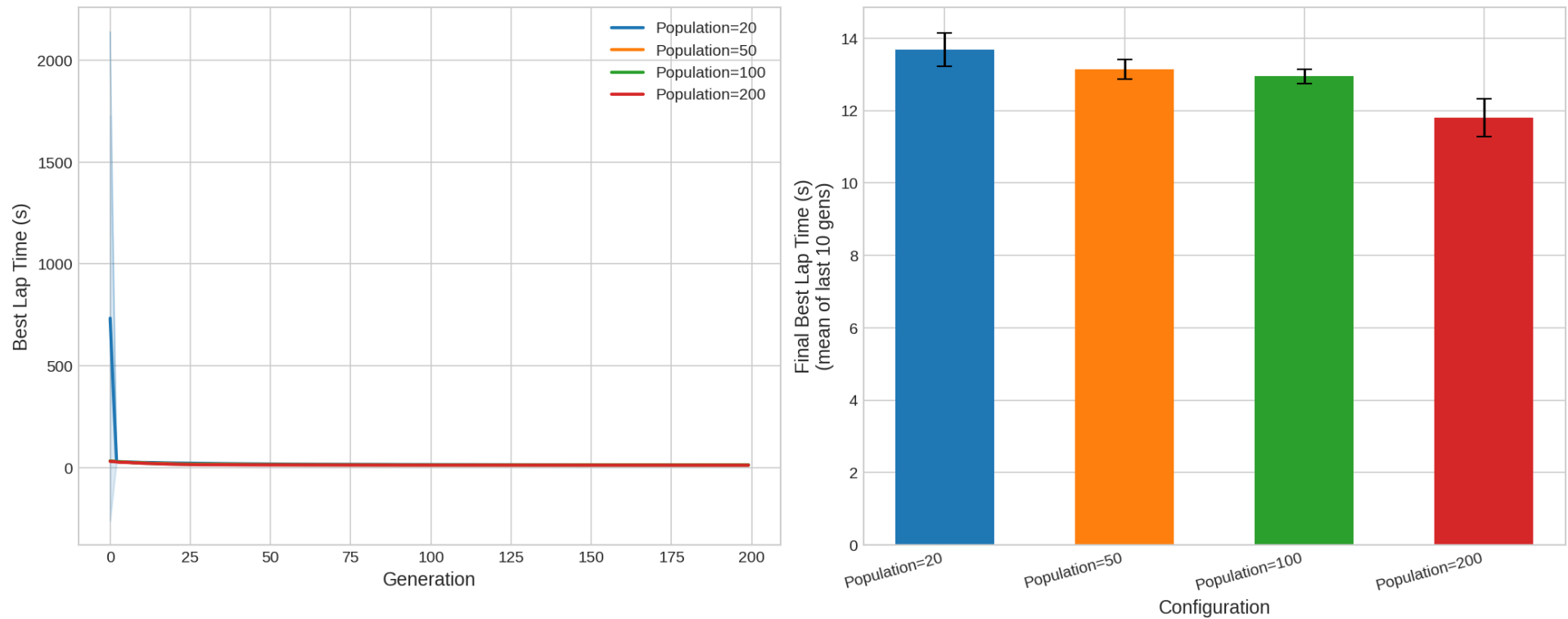
Result - Mutation Rate



A clear optimum at $p \approx 0.05$; high rates settle on a worse plateau they cannot escape.

Result - Population Size

Effect of Population Size on Fitness Quality



Final lap time falls monotonically: 13.7 s (20) → 11.8 s (200), but with diminishing returns per unit compute.

Conclusions

- The GA solves the racing-line problem effectively and stays fully legible - every generation is a drawable line.
- The penalty-based fitness function reliably enforces the on-track constraint within ~40 generations.
- Recommended configuration: multi-point crossover · mutation ≈ 0.05 · population 100–200.
- Crossover affects mostly consistency; mutation rate and population size most affect final quality.

Thank you

Questions & live demo

Oliwier Zasadni & Mikołaj Kimak